HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ASSIGNMENT 07

KNOWLEDGE BASE, KNOWLEDGE GRAPH, RAG VÀ AGENT

Sinh viên	Nguyễn Minh Vinh
Mã sinh viên	B23DCCN934
Lớp	D23CTPM01
Giảng viên	PGS.TS Trần Đình Quế

Hà Nội - 2026

MỤC LỤC NỘI DUNG

MỞ ĐẦU

1. Bối cảnh

2. Mục tiêu

3. Phạm vi

CHƯƠNG 1. CƠ SỞ TRI THỨC VÀ KNOWLEDGE GRAPH

1.1. Knowledge Base

1.2. Entity, relationship và attribute

1.3. Triple

1.4. RDF graph và property graph

1.5. Graph traversal

1.6. Ưu và nhược điểm

CHƯƠNG 2. RETRIEVAL-AUGMENTED GENERATION

2.1. Tại sao cần RAG

2.2. Indexing

2.3. Embedding và vector database

2.4. BM25

2.5. Retrieval, augmentation và generation

2.6. Cải tiến trước retrieval

2.7. Cải tiến sau retrieval

CHƯƠNG 3. GRAPHRAG VÀ AGENT

3.1. Giới hạn của Vector RAG

3.2. Graph retrieval

3.3. Kiến trúc GraphRAG

3.4. Vai trò Agent

3.5. Kiểm soát hallucination

CHƯƠNG 4. ỨNG DỤNG CHO BỆNH ĐÁI THÁO ĐƯỜNG

4.1. Bài toán

4.2. Hai dataset

4.3. Mô hình dự đoán và RAG

4.4. Code triển khai

4.5. Giao diện

4.6. Deploy

KẾT LUẬN

1. Kết quả đạt được

2. Hạn chế

3. Hướng phát triển

MỞ ĐẦU

1. Bối cảnh

Mô hình ngôn ngữ có khả năng tạo văn bản tự nhiên nhưng tri thức bên trong có thể lỗi thời, thiếu nguồn và phát sinh thông tin không có căn cứ. Retrieval-Augmented Generation (RAG) giải quyết một phần hạn chế này bằng cách truy xuất tri thức bên ngoài trước khi sinh câu trả lời. Knowledge Graph bổ sung cấu trúc quan hệ, đặc biệt hữu ích cho câu hỏi cần đi qua nhiều thực thể.

2. Mục tiêu

Bài tập xây dựng một trợ lý tri thức về đái tháo đường gồm Knowledge Base, Knowledge Graph, bộ truy xuất BM25 viết từ đầu, cơ chế GraphRAG đơn giản và web Agent có thể triển khai. Hệ thống phải trả lời kèm nguồn và không được trình bày đầu ra như chẩn đoán y khoa.

3. Phạm vi

Corpus gồm sáu đoạn tri thức được biên tập từ WHO, CDC và NIDDK, kèm mười bốn triple. Đây là tập minh họa nhỏ để làm rõ toàn bộ pipeline. Hệ thống chạy offline nhằm bảo đảm notebook và web có thể hoạt động khi không có API key hoặc dịch vụ vector database.

CHƯƠNG 1. CƠ SỞ TRI THỨC VÀ KNOWLEDGE GRAPH

1.1. Knowledge Base

Knowledge Base là tập thông tin được tổ chức để hệ thống truy vấn và sử dụng. Nguồn có thể là web, PDF, Word hoặc dữ liệu có cấu trúc. Mỗi tài liệu của bài lưu id, tiêu đề, URL và đoạn nội dung. URL giúp truy vết bằng chứng thay vì tạo câu trả lời không rõ nguồn.

1.2. Entity, relationship và attribute

Entity là đối tượng có định danh, ví dụ đái tháo đường, type 1, type 2 và HbA1c. Relationship biểu diễn liên hệ giữa hai entity như có loại, liên quan, có thể gây. Attribute là giá trị mô tả entity. Ba thành phần tạo nền tảng cho việc mô hình hóa tri thức.

1.3. Triple

Triple có dạng Subject - Predicate - Object. Ví dụ: type 2 - có yếu tố nguy cơ - ít vận động. Danh sách triple có thể được lưu dưới dạng JSON, RDF hoặc property graph. Cách biểu diễn này trực quan và hỗ trợ kiểm tra đường suy luận.

1.4. RDF graph và property graph

RDF chuẩn hóa tri thức thành các bộ ba và phù hợp liên kết dữ liệu trên web. Property graph cho phép node và cạnh mang thuộc tính, phù hợp truy vấn thực hành với Neo4j. Bài dùng JSON adjacency list để giữ môi trường gọn; cùng schema có thể nhập vào Neo4j khi cần mở rộng.

1.5. Graph traversal

Duyệt đồ thị bắt đầu từ entity xuất hiện trong câu hỏi, lấy các cạnh ở hop thứ nhất rồi tiếp tục tới hop thứ hai. Ví dụ, type 2 liên quan thừa cân; thừa cân có thể cải thiện bằng dinh dưỡng hợp lý. Đây là thông tin mà tìm kiếm từng đoạn độc lập có thể bỏ lỡ.

1.6. Ưu và nhược điểm

Knowledge Graph làm quan hệ rõ, hỗ trợ truy vấn multi-hop và giải thích đường đi. Nhược điểm là chi phí xây ontology, trích xuất entity, hợp nhất tên gọi và cập nhật dữ liệu. Graph sai có thể dẫn tới kết luận sai dù thuật toán truy vấn chính xác.

CHƯƠNG 2. RETRIEVAL-AUGMENTED GENERATION

2.1. Tại sao cần RAG

LLM thuần phụ thuộc tham số đã huấn luyện, có giới hạn thời điểm tri thức và có thể hallucinate. Fine-tuning thay đổi hành vi nhưng không phải cơ chế lưu trữ sự thật có thể cập nhật tức thời. RAG đưa bằng chứng liên quan vào context tại thời điểm hỏi và cho phép gắn nguồn.

2.2. Indexing

Indexing gồm thu thập, làm sạch, chia đoạn và lập chỉ mục. Chunk nhỏ tăng độ chính xác truy xuất nhưng có thể mất bối cảnh; chunk lớn giữ bối cảnh nhưng chứa nhiều nhiễu. Corpus nhỏ của bài dùng một chủ đề cho mỗi chunk; hệ thống lớn nên chia theo tiêu đề và dùng overlap.

2.3. Embedding và vector database

Dense embedding biến văn bản thành vector ngữ nghĩa; vector database lưu vector và tìm hàng xóm gần nhất theo cosine hoặc khoảng cách. Đây là hướng tốt cho truy vấn đồng nghĩa nhưng tăng dependency và chi phí. Bài chọn BM25 offline làm baseline có thể giải thích.

2.4. BM25

BM25 điều chỉnh điểm theo tần suất từ trong tài liệu, độ hiếm của từ trong corpus và độ dài tài liệu. Tham số k1 kiểm soát bão hòa tần suất; b kiểm soát mức chuẩn hóa độ dài. BM25 nhanh, không cần huấn luyện, nhưng nhạy với cách viết câu hỏi.

2.5. Retrieval, augmentation và generation

Retriever trả về top-k chunk có điểm cao. Augmentation ghép chunk vào context cùng câu hỏi và quy tắc trả lời. Generation tạo câu trả lời dựa trên context. Trong bản offline, generation là tổng hợp có kiểm soát; khi thay bằng LLM phải giữ quy tắc trích nguồn và từ chối khi thiếu bằng chứng.

2.6. Cải tiến trước retrieval

Query rewriting sửa câu hỏi mơ hồ; query expansion thêm từ đồng nghĩa; decomposition chia câu hỏi phức tạp thành câu hỏi con; HyDE sinh tài liệu giả định để truy xuất theo ý nghĩa. Các

kỹ thuật này tăng recall nhưng cũng có thể làm lệch ý định ban đầu.

2.7. Cải tiến sau retrieval

Reranking dùng mô hình chính xác hơn để sắp xếp lại ứng viên. Context compression bỏ phần ít thông tin nhằm giảm token và hiện tượng lost-in-the-middle. Hệ thống thực tế thường dùng retriever nhanh lấy nhiều ứng viên rồi reranker chọn ít đoạn tốt nhất.

CHƯƠNG 3. GRAPHRAG VÀ AGENT

3.1. Giới hạn của Vector RAG

Vector RAG mạnh với câu hỏi semantic nhưng mỗi chunk thường độc lập. Câu hỏi cần chuỗi quan hệ, chẳng hạn một loại bệnh liên quan yếu tố nào và yếu tố đó có thể cải thiện ra sao, cần kết hợp nhiều mẩu tri thức.

3.2. Graph retrieval

Graph retrieval nhận diện entity từ câu hỏi rồi duyệt các cạnh một hoặc nhiều hop. Kết quả là các đường có cấu trúc, giúp giải thích tại sao một thực thể liên quan thực thể khác. Với graph lớn cần giới hạn hop, loại cạnh và số nhánh để tránh bùng nổ.

3.3. Kiến trúc GraphRAG

Pipeline của bài chạy BM25 và graph traversal song song. Các đoạn văn cung cấp diễn giải và nguồn; graph cung cấp quan hệ. Agent hợp nhất hai loại context. Đây là GraphRAG mức cơ bản, chưa có community detection hoặc graph embedding.

3.4. Vai trò Agent

Agent là bộ điều phối: nhận câu hỏi, quyết định công cụ, lấy context, tạo câu trả lời và trả metadata. Tính agent không đồng nghĩa tự do thực hiện mọi hành động. Trong miền sức khỏe, công cụ và quyền phải giới hạn, có log nguồn và thông báo phạm vi.

3.5. Kiểm soát hallucination

Câu trả lời chỉ dùng nội dung truy xuất; nếu không có hit, hệ thống thông báo kho tri thức chưa đủ. Mỗi nguồn gồm tiêu đề, URL và điểm BM25. Knowledge Graph hiển thị đường liên kết. Các cơ chế này làm đầu ra có thể kiểm tra, dù không bảo đảm nguồn ban đầu đúng tuyệt đối.

CHƯƠNG 4. ỨNG DỤNG CHO BỆNH ĐÁI THÁO ĐƯỜNG

4.1. Bài toán

Người dùng đặt câu hỏi về dấu hiệu, yếu tố nguy cơ, xét nghiệm, biến chứng và phòng ngừa. Knowledge Base lấy thông tin giáo dục sức khỏe từ các cơ quan công. Hệ thống không phân loại type bệnh và không đưa chỉ định điều trị.

4.2. Hai dataset

Bài nộp kèm hai dataset tabular. `diabetes_dataset.csv` có 100.000 dòng và 16 cột, dùng cho công cụ MLP dự đoán nhãn diabetes. `AAPL_2015_2025.csv` có 2.766 phiên và 7 cột OHLCV, dùng minh họa dữ liệu chuỗi thời gian và công cụ phân tích chứng khoán. Ngoài ra, kho tri thức RAG có sáu chunk cùng mười bốn triple từ WHO, CDC và NIDDK; corpus này là dữ liệu bổ trợ, không thay thế hai dataset.

4.3. Mô hình dự đoán và RAG

Mô hình học máy nhận đặc trưng nhân khẩu và chỉ số sức khỏe để ước lượng nhãn thống kê. RAG giải thích khái niệm bằng tài liệu. Hai đầu ra có vai trò khác nhau: prediction không phải bằng chứng y khoa, còn tài liệu truy xuất không thay thế đánh giá cá nhân. Agent phải trình bày ranh giới này rõ ràng.

4.4. Code triển khai

Lớp BM25Retriever tự tính document frequency, IDF và điểm BM25. KnowledgeGraph tạo adjacency list và duyệt breadth-first tối đa hai hop. DiabetesAgent gọi hai thành phần, ghép bằng chứng và trả JSON gồm answer, sources, graph_paths, method và notice.

4.5. Giao diện

Web Flask có một ô câu hỏi, ba câu hỏi gợi ý và vùng kết quả. Mỗi nguồn là liên kết mở được cùng điểm BM25. API POST `/api/chat` phục vụ frontend; `/health` cho Render kiểm tra dịch vụ. Giao diện responsive và không lưu dữ liệu người dùng.

4.6. Deploy

Blueprint Render cài Flask và Gunicorn từ requirements.txt, sau đó chạy một worker bốn thread. Vì corpus và graph nằm trong repository, deploy không cần database, GPU hoặc API key. Cấu hình này phù hợp demo học tập trên gói miễn phí.

KẾT LUẬN

1. Kết quả đạt được

Bài đã xây dựng đủ Knowledge Base, Knowledge Graph, RAG scratch, GraphRAG mức cơ bản, Agent, ba notebook và web deploy. Các thành phần chạy offline và minh họa rõ từng bước từ indexing tới generation.

2. Hạn chế

Corpus nhỏ, chưa tự động ingest PDF/Word, chưa dùng embedding đa ngôn ngữ, reranker hoặc LLM. Entity matching hiện dựa token nên có thể bỏ sót biến thể ngôn ngữ. Nội dung chưa được thẩm định để sử dụng lâm sàng.

3. Hướng phát triển

Có thể thêm parser PDF/Word, BGE-M3, vector database, Neo4j, hybrid search, cross-encoder reranking, query decomposition và LLM có guardrail. Cần xây bộ câu hỏi keyword, semantic, single-hop và multi-hop để đánh giá retrieval độc lập với chất lượng generation.

TÀI LIỆU THAM KHẢO

[1] WHO, Diabetes fact sheet, https://www.who.int/news-room/fact-sheets/detail/diabetes.

[2] CDC, Diabetes Risk Factors; Diabetes Signs and Symptoms.

[3] NIDDK, Diabetes Tests & Diagnosis.

[4] Ngo Van Linh, Retrieval Augmented Generation, slide L6_LLMs-for-RAG-1.

[5] Tài liệu Knowledge Graph + RAG do người dùng cung cấp.

[6] S. Robertson và H. Zaragoza, The Probabilistic Relevance Framework: BM25 and Beyond, 2009.

[7] P. Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, 2020.